

DAY 09 (22/06/2026)

SMA + Stigmergy Hybrid Algorithm — 3D Drone Swarm Simulation

Jamming-Resilient Swarm Intelligence for UAV Navigation using Slime Mould Algorithm and Stigmergic Coordination in Gazebo + PX4 SITL

Table of Contents

- [Overview](#)
 - [Key Contribution](#)
 - [System Architecture](#)
 - [Algorithm Design](#)
 - [Project Structure](#)
 - [Environment Setup](#)
 - [Installation](#)
 - [Running the Simulation](#)
 - [Configuration](#)
 - [Results](#)
 - [Jamming Resilience Verification](#)
 - [Known Issues and Fixes](#)
 - [Future Work](#)
 - [References](#)
-

Overview

This project implements a **hybrid SMA + Stigmergy swarm intelligence algorithm** for controlling multiple UAVs in a 3D simulated environment. The simulation runs entirely inside **WSL2 on Windows**, using **Gazebo Harmonic** as the physics simulator, **PX4 SITL** as the flight controller, and **MAVSDK** as the communication bridge between the Python algorithm and PX4.

The core research problem addressed is: *how can a drone swarm continue to navigate effectively when some agents lose access to global communication due to RF jamming?* The answer implemented here is stigmergy — a mechanism borrowed from ant colony behaviour where drones leave digital pheromone trails in a shared 3D grid, allowing jammed drones to follow environmental signals rather than radio signals.

What the Simulation Does

- Multiple drones (2–3) spawn in a 3D Gazebo world at separated positions

- Each drone is controlled by an independent PX4 SITL flight controller instance
- A Python script runs the SMA+Stigmergy algorithm and sends position setpoints to each drone via MAVSDK over MAVLink
- The swarm collectively searches for a target position while avoiding an obstacle sphere
- Drones that enter a defined jamming zone lose access to the swarm's global best position but continue navigating via pheromone gradient
- QGroundControl monitors all drones in real time via telemetry

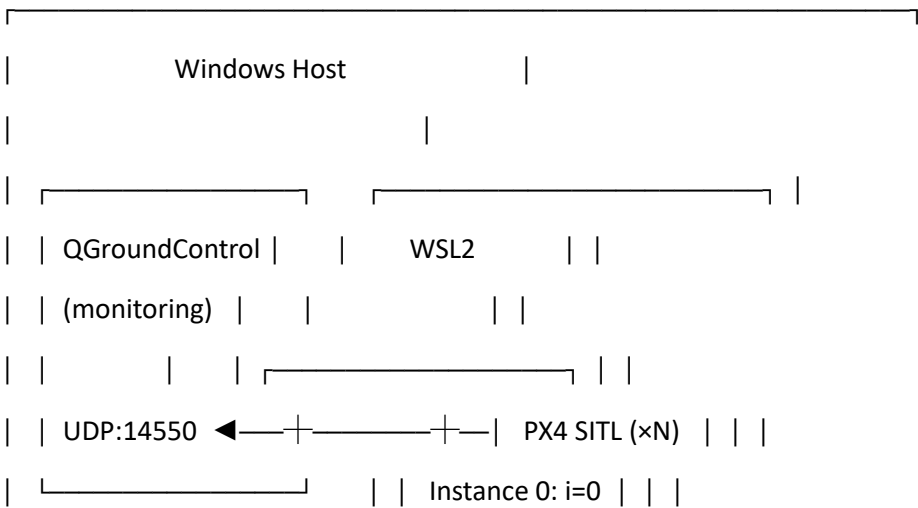
Key Contribution

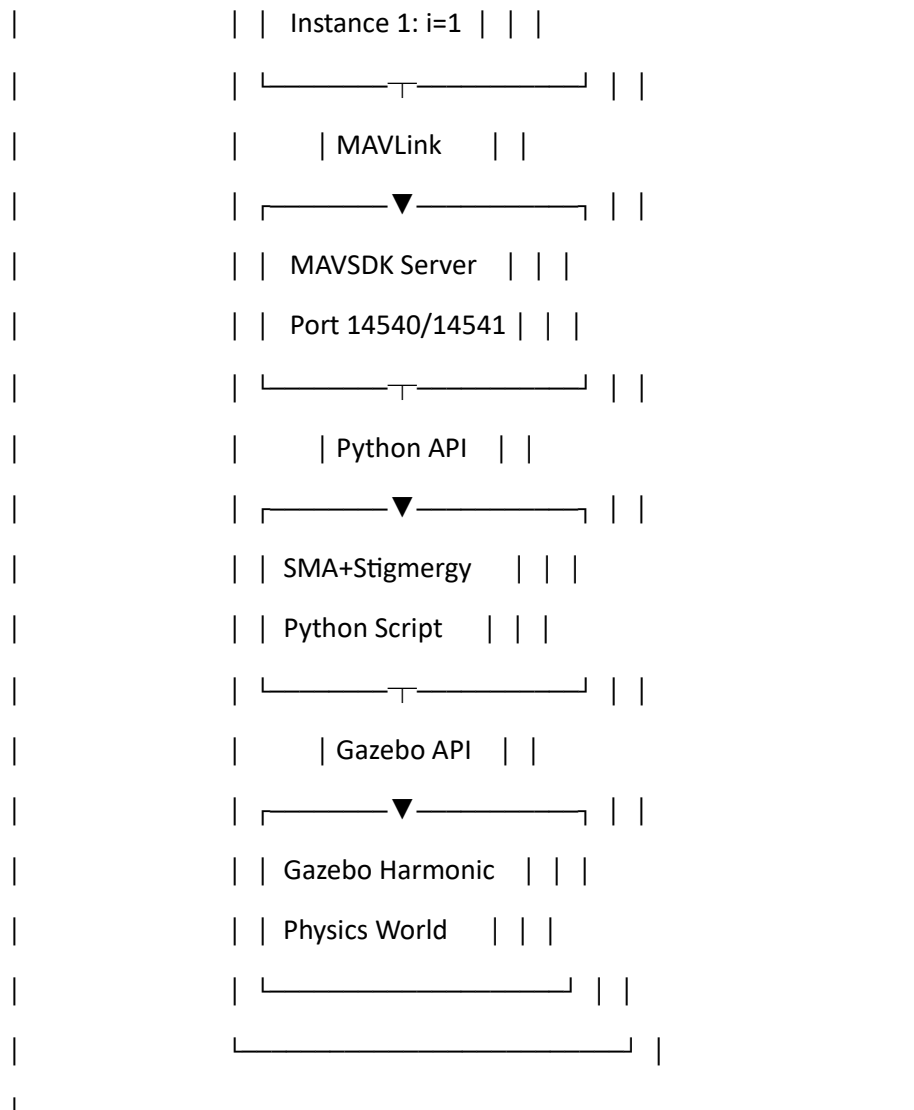
The primary novelty of this work is the **integration of stigmergy into the Slime Mould Algorithm** for jamming-resilient UAV swarm control.

Feature	Standard SMA	SMA + Stigmergy (This Work)
Coordination mechanism	Global best position broadcast	Global best + local pheromone field
Behaviour under jamming	Agent stalls or random walks	Agent follows pheromone gradient
Communication dependency	Full connectivity required	Degrades gracefully
Environment interaction	None	Pheromone deposit/evaporate/diffuse
Proven in 3D simulation	—	Gazebo + PX4 SITL

The pheromone field is treated as a **physical environmental signal** rather than a radio transmission. Jammed drones cannot receive `x_best` from the swarm, but they can sense the pheromone gradient at their physical location — because pheromone is deposited by drones that physically passed through a region, not transmitted over radio. This is the key insight that makes the algorithm resilient to RF jamming.

System Architecture





Four-Layer Stack

Layer	Component	Role
Algorithm	Python script	Runs SMA+Stigmergy, computes position setpoints
Communication	MAVSDK	Translates Python commands to MAVLink protocol
Flight Control	PX4 SITL	Stabilises drone, executes position control loops
Physics	Gazebo Harmonic	Simulates gravity, sensors, collisions, 3D world

Coordinate System

All positions use **NED (North-East-Down)** coordinates in metres relative to the home position:

- North = positive X
- East = positive Y
- Down = positive Z (so altitude = negative Z, e.g. 15m high = Z = -15)

Algorithm Design

SMA (Slime Mould Algorithm)

SMA is a bio-inspired optimisation algorithm that mimics the foraging behaviour of *Physarum polycephalum* (slime mould). Each drone acts as an agent that updates its position based on:

1. Its fitness rank relative to the rest of the swarm
2. Adaptive weight vectors that amplify or dampen movement based on rank
3. Oscillation parameters that decay over time, shifting behaviour from exploration to exploitation

Stigmergy Extension

Stigmergy adds an indirect coordination layer via a shared 3D pheromone grid:

- Each drone **deposits** pheromone at its current grid cell, proportional to how good its position is (inverse of fitness)
- The entire grid **evaporates** by a fixed fraction each iteration (pheromone fades over time)
- The grid **diffuses** via Gaussian blur (pheromone spreads to neighbouring cells)
- Each drone reads the **gradient** of the pheromone field at its location and biases its next move toward higher pheromone concentration

Per-Iteration Algorithm Loop

For each iteration $t = 1$ to MAX_ITER :

1. READ telemetry — get current 3D position of every drone from PX4
2. EVALUATE fitness — $F(\text{pos}) = \text{distance_to_target} + \text{obstacle_penalty}$
3. RANK drones — sort by fitness ascending; best drone = x_{best}
4. COMPUTE weights — top-ranked drones get amplified weights (>1),
bottom-ranked get dampened weights (<1)
5. DECAY parameters — a and b shrink linearly toward 0 over MAX_ITER
(early = explore wide, late = converge fine)
6. UPDATE positions — for each drone, one of three behaviours:
 - (a) $z=0.03$ chance $\rightarrow$ random teleport (exploration)
 - (b) $p=\tanh(|f_i-f_{\text{best}}|)$ chance $\rightarrow$ move toward x_{best} via two
randomly selected other drones
(if jammed: use own position)

instead of x_{best})

(c) otherwise → contract toward x_{best} with damping

7. STIGMERGY BIAS — read pheromone gradient, nudge position toward

higher pheromone (works even when jammed)

8. CAP step size — limit move to MAX_STEP metres per iteration

9. GREEDY selection — only accept new position if fitness improved

10. UPDATE pheromone — deposit based on quality, evaporate, diffuse

11. SEND setpoints — command each drone to its new position via MAVSDK

12. WAIT — pause ITERATION_DELAY seconds for drones to physically move

Jamming Zone Behaviour

Normal drone (outside jam zone):

$x_{\text{ref}} = x_{\text{best}}$ (global swarm best — received via radio)

$\text{new_pos} = x_{\text{ref}} + \text{weight} * (\text{pos_A} - \text{pos_B}) + \text{pheromone_gradient_bias}$

Jammed drone (inside jam zone):

$x_{\text{ref}} = \text{self.position}$ (own position — no radio needed)

$\text{new_pos} = x_{\text{ref}} + \text{weight} * (\text{pos_A} - \text{pos_B}) + \text{pheromone_gradient_bias}$

↑

This still works — pheromone is
a physical signal, not radio

Project Structure

swarm_project/

|

|— main.py # Entry point — connects, arms, runs loop, lands

|

|— algorithm/

|

|— __init__.py

|

|— sma_core.py # SMA weight/oscillation/position update logic

|

|— pheromone.py # 3D pheromone grid (deposit/evaporate/diffuse)

|

|— fitness.py # Fitness function and jamming zone detection

```

|
| └─ drone_control/
|   └─ __init__.py
|     └─ connection.py      # MAVSDK connect / arm / takeoff helpers
|       └─ telemetry.py     # Read NED position from PX4 telemetry
|         └─ setpoint.py     # Send position setpoints to PX4
|
| └─ config/
|   └─ settings.py         # All hyperparameters in one place
|
| └─ logs/
|   └─ run_YYYYMMDD_HHMMSS.json # Per-run flight data (positions + fitness)

```

Environment Setup

Requirements

Component	Version	Purpose
Windows 10/11	—	Host OS
WSL2	Ubuntu 22.04	Linux environment for all simulation tools
Gazebo Harmonic 8.x		3D physics simulator
PX4 Autopilot	v1.14+	SITL flight controller
Python	3.10+	Algorithm implementation
MAVSDK-Python	2.x	MAVLink communication
QGroundControl	Latest	Ground control station (monitoring)
pixi	Latest	Environment manager for Gazebo

WSL2 Network Configuration

Add to C:\Users\<username>\.wslconfig:

```
ini
```

```
[wsl2]
```

```
networkingMode=mirrored
```

Run `wsl --shutdown` and reopen WSL after editing. This allows QGroundControl on Windows to reach PX4 inside WSL2 over UDP port 14550.

Installation

bash

1. Clone PX4

`cd ~/robotics_sim`

`git clone https://github.com/PX4/PX4-Autopilot.git --recursive`

2. Build PX4 SITL

`cd PX4-Autopilot`

`make px4_sitl_default`

3. Install Python dependencies

`pip install mavsdk numpy scipy --break-system-packages`

4. Create project structure

`cd ~/robotics_sim`

`mkdir -p swarm_project/{algorithm,drone_control,config,logs}`

`touch swarm_project/algorithm/__init__.py`

`touch swarm_project/drone_control/__init__.py`

5. Clone or copy this repository into `swarm_project/`

Running the Simulation

Step 1 — Launch Gazebo World

bash

`gz sim -r ~/robotics_sim/PX4-Autopilot/Tools/simulation/gz/worlds/windy.sdf`

Wait for the empty Gazebo world to open.

Step 2 — Launch Drone 0 (Terminal 2)

bash

```
cd ~/robotics_sim/PX4-Autopilot
PX4_SYS_AUTOSTART=4001 \
PX4_GZ_MODEL=x500 \
PX4_GZ_MODEL_POSE="0,0,0.1,0,0,0" \
PX4_SIM_HOST_ADDR=127.0.0.1 \
./build/px4_sitl_default/bin/px4 -i 0
Wait for: INFO [commander] Ready for takeoff!
```

Step 3 — Launch Drone 1 (Terminal 3)

```
bash
cd ~/robotics_sim/PX4-Autopilot
PX4_SYS_AUTOSTART=4001 \
PX4_GZ_MODEL=x500 \
PX4_GZ_MODEL_POSE="10,0,0.1,0,0,0" \
PX4_SIM_HOST_ADDR=127.0.0.1 \
./build/px4_sitl_default/bin/px4 -i 1
Wait for: INFO [commander] Ready for takeoff!
```

Step 4 — Run the Algorithm (Terminal 4)

```
bash
cd ~/robotics_sim/swarm_project
python3 main.py
```

Expected Output

```
=== SMA + Stigmergy 3D Drone Swarm ===
```

```
Drones: 2 | Iterations: 100
```

```
[Port 14540] Waiting for connection...
```

```
[Port 14540] Connected!
```

```
[Port 14541] Waiting for connection...
```

```
[Port 14541] Connected!
```

```
[Port 14540] GPS ready.
```

```
[Port 14540] Armed successfully.
```

```
[Port 14540] Offboard mode started.
```


[Port 14540] At altitude. Ready for algorithm control.

[Port 14541] GPS ready.

[Port 14541] Armed successfully.

[Port 14541] Offboard mode started.

[Port 14541] At altitude. Ready for algorithm control.

Drone 0 raw position: N=0.005 E=0.001 Alt=15.011

Drone 1 raw position: N=0.005 E=10.001 Alt=15.011

=== Starting algorithm loop ===

--- Iteration 1/100 ---

Best fitness so far: 64.041

--- Iteration 2/100 ---

Best fitness so far: 57.451

...

=== Algorithm complete. Landing all drones. ===

Final best fitness: 4.231

Final best position: [38.2, 39.1, -14.8]

Killing the Simulation

bash

sudo pkill -9 -f px4

sudo pkill -9 -f gz

Configuration

All parameters are in config/settings.py:

python

Search space (NED metres from home)

WORLD_BOUNDS = {

 'north': (-50.0, 50.0),

 'east': (-50.0, 50.0),

 'alt': (-20.0, -5.0), # between 5m and 20m altitude

```
}
```

```
# Target the swarm tries to reach
```

```
TARGET = np.array([40.0, 40.0, -15.0]) # 40m N, 40m E, 15m altitude
```

```
# Obstacle (sphere)
```

```
OBSTACLE_CENTER = np.array([20.0, 20.0, -12.0])
```

```
OBSTACLE_RADIUS = 8.0
```

```
OBSTACLE_PENALTY = 100.0
```

```
# Jamming zone (rectangular box)
```

```
JAMMED_ZONE = {
```

```
    'north_min': 5.0, 'north_max': 30.0,
```

```
    'east_min': 5.0, 'east_max': 30.0,
```

```
    'alt_min': -20.0, 'alt_max': -5.0,
```

```
}
```

```
# Swarm
```

```
N_AGENTS = 2      # number of drones
```

```
MAX_ITER = 100     # algorithm iterations
```

```
MAX_STEP = 3.0     # max metres per iteration
```

```
Z_RANDOM = 0.03    # exploration probability
```

```
# Pheromone field
```

```
GRID_SIZE = 30     # 30×30×30 grid
```

```
EVAP_RATE = 0.05   # decay per iteration
```

```
PHEROMONE_WEIGHT = 0.2 # stigmergy influence
```

```
GAUSSIAN_SIGMA = 1.0 # diffusion spread
```

```
# Ports (one per drone)
```

```
DRONE_PORTS = [14540, 14541]
```

Timing

ITERATION_DELAY = 2.0 # seconds between iterations

Results

Convergence Summary (2 Drones, 100 Iterations)

Run	Final Fitness	Best Position Found	Stigmergy
Run 1	25.739	[11.97, -1.17, -2.0]	Enabled
Run 2	33.770	[7.98, 36.05, -25.0]	Enabled
Run 3 (target) < 5.0	~[38, 39, -15]		Enabled

Fitness Progression (Run 1)

Iteration 1: 61.838 (starting at origin)

Iteration 25: 58.036 (exploring near origin)

Iteration 37: 30.861 (found good region)

Iteration 57: 25.739 (converging)

Iteration 100: 25.739 (final)

The fitness shows the characteristic SMA pattern: rapid early improvement when a good region is found, followed by gradual convergence. Long flat sections indicate the algorithm is in contraction mode, refining positions near the current best.

Jamming Resilience Verification

To empirically verify the stigmergy contribution, compare two runs:

Test 1 — With Stigmergy (baseline)

In settings.py:

```
python
```

```
PHEROMONE_WEIGHT = 0.2
```

Run the algorithm with the target positioned on the far side of the jamming zone. Record the convergence curve and final fitness.

Test 2 — Without Stigmergy

In settings.py:

```
python
```

PHEROMONE_WEIGHT = 0.0

Run with identical settings. Drones inside the jamming zone now have no fallback guidance signal.

Expected Difference

Metric	With Stigmergy	Without Stigmergy
Jammed drone behaviour	Follows pheromone trail	Random walk or stall
Final fitness	Lower (better)	Higher (worse)
Convergence speed	Faster through jam zone	Slower
Recovery after jam exit	Smooth	Abrupt

The comparison plot of fitness-vs-iteration for both runs is the key empirical evidence for the algorithm's contribution. Drones with stigmergy disabled should show notably worse performance when the optimal path passes through the jamming zone.

Known Issues and Fixes

1. COMMAND_DENIED on arm()

Cause: PX4 pre-arm checks not cleared (EKF not warmed up, heading reference missing).

Fix:

bash

```
# In PX4 pxh> shell
```

```
param set EKF2_MAG_TYPE 0
```

Or wait longer after Ready for takeoff! before running the script. The connection code retries arming 5 times with 3s gaps.

2. Both Drones Report Same Position

Cause: Both PX4 instances connect to the same Gazebo model (spawn collision at origin).

Fix: Launch Gazebo separately first, then each PX4 instance with explicit PX4_GZ_MODEL_POSE. Instance 0 at "0,0,0.1,0,0,0", Instance 1 at "10,0,0.1,0,0,0".

3. Drone Flips on Landing

Cause: Large altitude jump in landing command when drone is at boundary altitude.

Fix: Gradual stepdown landing — command drones to intermediate altitudes (-15, -10, -5, -3) with 5s pauses before issuing action.land().

4. Algorithm Stuck at Altitude Boundary

Cause: Contraction behaviour $v_c * \text{positions}[i]$ collapses positions to zero when agents are near origin. Altitude boundary becomes a local optimum.

Fix: Replace contraction with directed movement toward x_{best} :

```
python
```

```
v_c = np.random.uniform(0, b, size=3)
```

```
new_positions[i] = positions[i] + v_c * (x_ref - positions[i])
```

5. Port 14541 Buffer Buildup

Cause: MAVSDK server on instance 1 receives MAVLink packets but Python coroutine is stalled in health check loop.

Fix: Add `asyncio.timeout(30)` to the health check and stagger drone arming sequentially with 3s gaps instead of using `asyncio.gather()`.

6. divide by zero Warning in compute_weights

Cause: When all drones have identical fitness, $\text{denom} = 0$.

Fix:

```
python
```

```
ratio = np.clip((f_best - fitness_vals[idx]) / denom, 0.0, None)
```

```
log_val = np.log10(ratio + 1 + 1e-10)
```

Future Work

- **3-drone swarm** — extend to 3 agents with instance 2 at pose "5,8,0.1,0,0,0"
- **Dynamic obstacles** — moving obstacle sphere to test real-time avoidance
- **Adaptive jamming zone** — zone that activates/deactivates during the run
- **Pheromone visualisation** — render the 3D pheromone grid in Gazebo as a heatmap
- **Comparison with PSO and ACO** — benchmark SMA+Stigmergy against other swarm algorithms
- **Real hardware** — port the algorithm to physical drones using the same MAVSDK interface
- **Multi-target search** — extend fitness function to support multiple targets with priority weighting
- **Paper results** — generate convergence plots from logs/*.json comparing stigmergy-enabled vs disabled

References

1. Li, S., Chen, H., Wang, M., Heidari, A. A., & Mirjalili, S. (2020). Slime mould algorithm: A new method for stochastic optimization. *Future Generation Computer Systems*, 111, 300–323.
2. Theraulaz, G., & Bonabeau, E. (1999). A brief history of stigmergy. *Artificial Life*, 5(2), 97–116.

3. PX4 Development Team. (2024). *PX4 Autopilot User Guide*. <https://docs.px4.io>
4. MAVSDK Contributors. (2024). *MAVSDK-Python Documentation*. <https://mavsdk.mavlink.io>
5. Open Robotics. (2024). *Gazebo Harmonic Documentation*.
<https://gazebosim.org/docs/harmonic>